

인공지능 과제 4 보고서

고양이가 있는 스마트홈 로봇청소기 MDP 구현

과목명

인공지능

분반

4분반

이름

양현성

학번

202412999

제출일

2026.6.14

담당교수

김성찬

1. 과제 개요

이번 과제는 여러 방해 요소와 두가지의 변수가 존재하는 환경에 대한 상태 공간의 탐색과 **MDP**를 활용한 가치 탐색의 과제이며, 여러 선택 가능한 행동이 있을 때, 해당 행동에 대해서 자율적인 탐색을 가능케 하고 그리디한 행동을 결정함에 있어 이는 **MDP** 문제라 볼 수 있다. **Value Iteration**과 **Policy Iteration**은 둘 다 같은 최적 정책에 대한 방법이지만, 해당 방식은 서로 전체 정책에 대한 보상을 고려하는지, 혹은 해당 정책에 대하여 작은 단위에서의 수렴을 고려하는지에 따라 다르기 때문이다.

2. 실험 환경

운영체제

Debian GNU/Linux 12 (bookworm)

Python 버전

Python 3.11

실행 명령

CLI: `uv run python main_cli.py`

WEB: `uv run python main_web.py`

생성형 **AI** 사용 여부

사용 / 미사용 (해당 항목에 표시)

3. MDP 모델링

3-1. 상태(State) 정의

상태는 현재 갈 수 있는 위치, 현재 남은 배터리, 그리고 먼지의 존재 여부 4가지를 포함하고 있다. 위치는 이후 맵에서 갈 수 있는 좌표들의 모음이기 때문에 에이전트가 행동을 결정하여 맵과 상호작용하기 위해 필수적이며, 배터리는 에이전트에게 제한 사항을 걸어 특정 움직임을 최적화하는 목적이고, 먼지 존재 여부는 에이전트가 현재 이 위치를 타깃으로 움직여야 함을 상기하게 한다. 상태 공간에 대한 크기는 위치의 갯수에 영향을 받게 되는데, 이는 맵의 크기와 맵 내에서 벽으로 막힌 공간 등의 영향을 받게 된다.

3-2. 행동(Action) 및 종료 조건

행동은 앞, 뒤, 좌, 우, 청소기 가동, 배터리 충전 등의 행위를 할 수 있으며, 모든 먼지가 제거되거나, 배터리가 0이 되거나, 혹은 종료 위치에 있을 시 **terminal state**에 도달하게 된다.

3-3. 전이(Transition) 모델

일반 이동은 배터리 상태를 1 소모하고 다음 위치로 이동하여 전이할 수 있으며, 청소할 시 빈 바닥을 청소하거나 먼지가 있는 바닥을 청소하도록 전이할 수 있다. 충전은 베이스 스테이션 위치에 도달했을 경우 배터리를 무조건 충전하도록 상태가 전이한다. 그 후 다시 더러워질 경우에 대한 상황을 실현한 뒤 해당 상태에 대해서 다시 전이를 시도한다.

3-4. 보상(Reward) 설계

기본 이동

모든 이동에 있어서는 -1 패널티를 부여한다.

모든 이동에는 배터리를 소모하므로, 최대한 많은 먼지를 제거해야 하는 시나리오에서 움직임을 최소한으로 해야 하기 때문에 패널티를 부여해야 옳다.

벽 충돌 / 고양이 간섭

벽 충돌에 대해선 -10 패널티 부여, 고양이 충돌에 대해선 -5 패널티를 부여한다. 모든 충돌과 간섭은 무조건 피해야 하므로 에이전트에게 강한 패널티를 주는 것이 옳다.

청소 성공

청소를 성공하였을 경우에는 무조건 100 보상을 부여한다. 에이전트는 배터리가 부족하지 않은 이상 청소를 우선으로 움직여야 하기 때문에 청소를 할 시 100보상을 부여하여 큰 보상을 부여한다.

빈 바닥 청소 / 충전 실패

둘다 -10의 패널티를 부여한다. 해당 행동은 배터리를 과도하게 소모하는 행위가 되기 때문에 최대한 에이전트의 행동 전이에서 제거해야 하는 행위이기 때문에 패널티를 부여해야 옳다.

모든 먼지 청소 / 복귀 성공

모든 청소를 마칠 때는 **300** 보상 부여, 복귀까지 성공했을 경우 **1000**보상을 부여한다.

모든 청소를 마치는 것이 중요하기 때문에 보상을 **300** 부여하지만, 무엇보다 배터리가 모두 다 되어 복귀하지 못하는 경우가 제일 치명이므로 무조건 복귀하는 시나리오 하에서 **1000** 보상을 부여하는 것이 옳다.

4. Task 수행 요약

1

상태 공간 생성

주어진 가로 세로 높이를 모두 순회하며 벽면을 제외한 모든 좌표를 생성한다.

2

종료 조건 정의

배터리가 **0** 이하인지 검사하고, 더 이상 맵에 먼지가 없고 현재 위치가 종료 위치인지 검사한다.

3

전이 확률 구현

현재 상태에 따라 배터리 충전 가능 여부 확인, 청소를 시작할 시에 해당 위치가 실제로 먼지가 있는지 확인, 이동할 시에 해당 위치가 이동 가능한지 확인한 뒤 보상을 계산하고 다시 더러워질 확률을 곱하여 최종 확률을 구한다.

4

보상 함수 구현

위, 아래, 좌, 우, 청소, 충전, 고양이와 벽과의 충돌, 이동 시 고양이 혹은 젖은 지역에 들어갔을 시 점수를 부여한다.

5

Value Iteration

벨만 방정식을 이용하여 모든 후보군 상태에 대해 **Q-value**를 구하고 정해진 양의 반복이 끝날때까지 반복 후 최대 **Q-value**를 받은 행동을 저장한다.

6

Policy Extraction

에이전트가 현재 처해진 환경에서 취할 수 있는 모든 상태에 대해 **Q-value**를 구하고, 해당 행동별로 가장 높은 **Q-value** 값을 비교해 가장 높은 **Q-value**를 받은 행동을 저장한다.

7

Q-value 계산

모든 전이 가능한 시나리오에 대해서 **Q-value** 공식에 따라 $\text{확률} * \text{보상} + \text{감마} * \text{다음 상태}$ 를 계산한다.

8

Policy Evaluation

현재 주어진 정책에 대해 해당 상태에서 수행할 시 **Q-value**를 적용해 기댓값까지 수렴하도록 반복한다.

9

Policy Iteration

최대 반복 제한동안 모든 정책에 대한 가치를 초기값으로 설정한 뒤, 모든 정책의 평가를 실행하고, 그 후 받은 **Q-value**를 바탕으로 모든 가능한 방향의 정책을 대입한 후 가장 높은 값을 받은 정책들로 계속 다음 정책을 그리디하게 선택하여 정책이 굳을때까지 반복한다.

5. 실행 및 검증 결과

CLI 실행

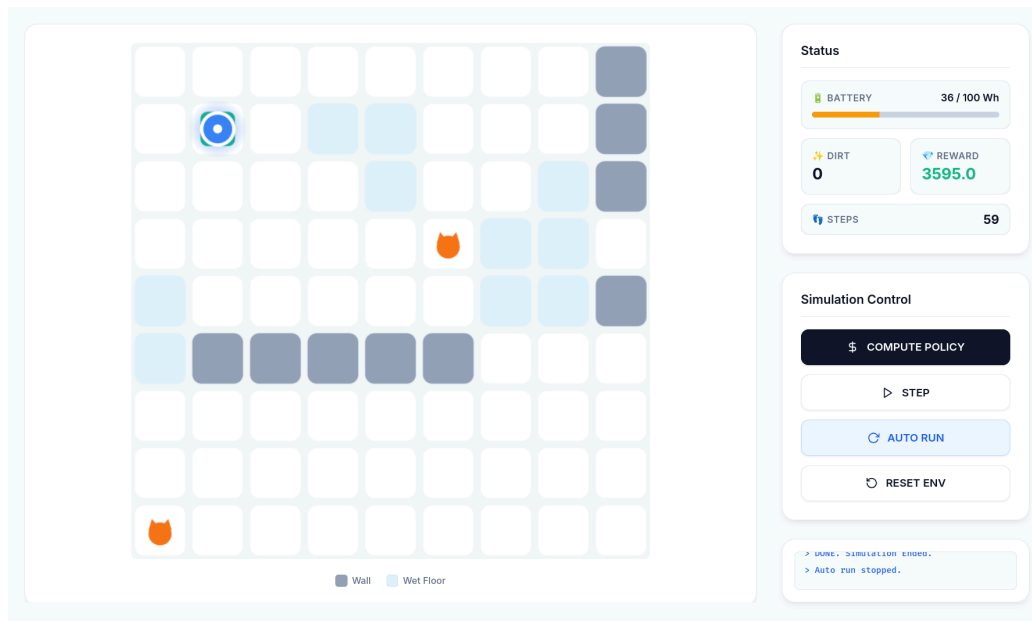
`main_cli.py`를 실행하였으며, 모든 청소를 성공하고 최종 **2439**점을 얻으며 해당 실행을 마무리했다. 아직 젖은 바닥에 대한 역치가 너무 낮으며, 모든 먼지를 청소하는 경우에 대해서 높은 확률로 성공함이 보였다.

```
+-----+
| . . . . . |
| . . . . . |
| . . . . . |
| . ### . . [K] . . |
| . ### . . . . . |
| ##### ~ ~ ##### |
| . ### ~ ~ ~ . . . |
| [K]### ~ [R] . . [K] . |
+-----+

[R]:Robot | [B]:Base Station | [K]:Cat | *:Dirt | ~:Wet | ###:Wall
Final Reward: 2439.0
○ (cat-robot-mdp-starter) maka@mk-cilab:~/Workspace/hw3$
```

웹 GUI 정책 계산 / AUTO RUN 동작

`main_web.py`를 실행하였으며, 모든 청소를 성공하고 최종 **3595**점을 얻으며 해당 실행을 마무리했다. **59**스텝을 완수하였으며, 높은 확률로 모든 배터리를 소진하지 않고 모든 먼지를 제거하는데 성공하였다.



Policy Iteration 실행

tools/verify_performance.py 를 실행하였으며, 10번의 모든 루프 끝에 100%의 성공률과 평균 보상값 1968점, 평균 스텝 54스텝을 기록하였다.

```
(cat-robot-mdp-starter) maka@mk-cilab:~/Workspace/hw3$ uv run python tools/verify_performance.py
Trial 1/10...
  Success: True, Reward: 1916.0, Steps: 44
Trial 2/10...
  Success: True, Reward: 2144.0, Steps: 79 ...
Trial 5/10...
  Success: True, Reward: 1837.0, Steps: 35
Trial 6/10...
  Success: True, Reward: 1474.0, Steps: 16
Trial 7/10...
  Success: True, Reward: 1836.0, Steps: 50
Trial 8/10...
  Success: True, Reward: 2302.0, Steps: 76
Trial 9/10...
  Success: True, Reward: 2084.0, Steps: 75
Trial 10/10...
  Success: True, Reward: 2688.0, Steps: 105

Results:
  Success Rate: 100.0%
  Average Reward: 1968.0
  Average Steps: 54.9
(cat-robot-mdp-starter) maka@mk-cilab:~/Workspace/hw3$
```

6. Value Iteration과 Policy Iteration 비교

구현 난이도

계속 생성된 정책이 기존 정책과 달라진 점이 있는지에 대해 비교해야 하고, 초기화된 정책을 계속 업데이트 해야 하는 점에서 정책 반복이 구현 난이도가 더욱 높다.

수렴 과정 특징

가치 반복은 특정한 컷오프 값에 도달해 수렴할때까지 단순히 해당 상태와 행동에 대한 모든 조합을 시도해보며 단순 반복하지만, 정책 반복은 정책에 대해서 계속 정책들을 세세하게 바꿔가며 가장 좋은 정책 하나로 수렴할때까지 반복하게 된다.

체감 속도

각 상태에 대해서 6가지의 행동만 계산하면 되는 가치 반복이 모든 상태와 전체 선택에 대해서 계산하는 정책 반복에 비해 매우 빠르다.

정책 품질 / 차이

정책이 완전히 굳을 때에만 해당 정책을 시행하는 정책 반복과 달리 가치 반복은 특정 컷오프만 만족하면 수렴하므로 가장 최적의 행동을 선택하지 못하는 경우가 존재할 수 있다.

7. 분석 및 고찰

해당 상태 공간에서는 두가지 요소가 상태 공간을 가장 키우게 되는데, 남은 배터리 양과 먼지가 위치하는 갯수와 여부가 모든 좌표점에 대하여 존재할 수 있기 때문에, 해당 두 값의 영역의 곱과 맵의 모든 좌표점 갯수의 곱만큼 상태 공간이 비대해지게 된다.

고양이 간섭과 젖은 바닥 중에선 고양이의 간섭을 가장 피하려는 양상을 크게 보였으며, 내부적으로 게임을 종료할 수 있는 가장 짧은 수를 발견한다면 젖은 바닥의 간섭을 감수하는 행동을 보였다.

움직임에 대한 패널티보다 젖은 바닥에 대한 패널티를 더욱 높게 설정할 시, 젖은 바닥을 더 이상 밟지 않고 최대한 마른 바닥을 움직이려고 시도할 것이다.

고양이가 움직일 경우, 고양이 근처에서 먼지가 생성될 가능성이 높는데, 이 경우 고양이가 움직이는 확률까지 포함하여 모든 위치좌표에 대하여 고양이에 대한 정보를 저장해야 하므로 극심한 상태 공간 폭발이 발생할 가능성이 높다.

8. 생성형 AI 사용 공개

사용 도구

Google Gemini

사용한 범위

벨만 방정식 구현 및 검증, 초기 소스 분석

직접 검증한 내용

모든 코드에 대한 작동 원리, 보상값, 정책 생성
코드에 대한 유효성 검증

AI 제안을 수정한 부분

해당사항 없음. 내 코드를 AI의 제안으로
수정.

9. 결론

해당 과제를 통하여 세가지를 깨달을 수 있었다. 상태 공간의 매개변수 갯수에 따라 달라지는 상태 공간의 크기는 최적화와 직결적으로 연관된다는 점, 보상 함수에 대한 값 설정의 미비는 에이전트의 그리디한 결정에 있어 도움이 되지 않는다는 점, 에이전트의 정책을 결정하는데 있어 최상의 결과를 항상 기대할수 없다는 점을 알 수 있었다.

10. 제출 전 자가 점검

- student/task1 ~ task9 구현 완료
- CLI / 웹 GUI 검증 완료
- 보고서 PDF 변환 완료
- AI 사용 공개 작성 완료
- 압축파일명 형식 준수 완료